

0.109. La fonction `changerDessin()` — redessiner une tuile partout

`#include <changerDessin>` : `changerDessin(tuile, dessin)` change le dessin d'une tuile, et toutes les cases qui la portent changent d'un coup.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `changerDessin(tuile, dessin)` remplace le **dessin** d'une tuile dans la mémoire vidéo. Toutes les cases qui portent cette tuile changent **en même temps**, sans en poser une seule : c'est ainsi qu'on anime l'eau, le feu, l'herbe.

La ligne à écrire : `#include <changerDessin>`. Ici aussi `<Tuile>` : on dessine deux tuiles.

Ses arguments : la tuile à changer, puis le dessin à lui donner.

Ce qu'elle coûte : environ **830 octets** : elle lit les dessins à leur place dans la cartouche, et le compilateur grave la police entière.

Essaie : `changerDessin(EAU, EAU);` rend le dessin d'origine.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1022 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	52 o	← la console, à l'allumage – la mise en route de la console
main()	150 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	736 o	← changerDessin() – la police entière, et 2 tuiles dessinées

Lettres gravées : toutes (`changerDessin()` lit les dessins à leur place)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>changerDessin()</code>	× 2	88 o sur place, dans main() + Tuiles (736 o)
<code>poser()</code>	× 3	18 o sur place, dans main()
<code>images()</code>	× 3	6 o sur place, dans main()
<code>image()</code>	× 1	3 o sur place, dans main() + AttendreImage (28 o)

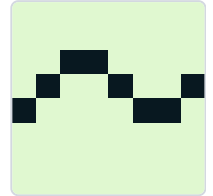
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <changerDessin> : redessiner une tuile partout ----
// Trois cases d'EAU. Toutes les 30 images, la tuile EAU prend le dessin
// VAGUE, puis le sien : les trois cases bougent ensemble.

#include <Tuile>          // pour dessiner des tuiles
#include <poser>
#include <changerDessin> // changerDessin() : un autre dessin pour une tuile
```

2. UN DESSIN — EAU

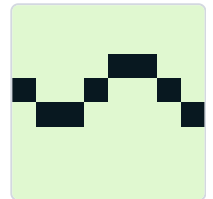
```
Tuile EAU = {
    ".....", ".....", "..##....", ".#..#..#",
    "#...##.", ".....", ".....", ".....",
};
```



à l'écran

3. UN DESSIN — VAGUE

```
Tuile VAGUE = {
    ".....", ".....", "...##..", "#..#..#",
    ".##....#", ".....", ".....", ".....",
};
```



à l'écran

4. UNE VARIABLE — VAGUE

```
uint8_t vague = 0; // 1 : la tuile EAU porte le dessin VAGUE
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(4, 8, EAU);
    poser(5, 8, EAU);
    poser(6, 8, EAU);

    while (true) {
        image();
        if (images() == 30 || images() == 90 || images() == 150) { // de temps en temps
            if (vague == 0) {
                changerDessin(EAU, VAGUE); // les trois cases d'un coup
                vague = 1;
            } else {
                changerDessin(EAU, EAU); // le dessin d'origine
                vague = 0;
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

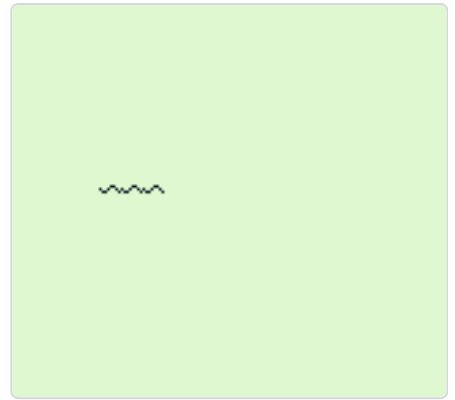
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Trois vagues qui changent de forme ensemble, de temps en temps.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les trois cases portent la même tuile
- ✓ le dessin de la tuile a changé
- ✓ et les cases n'ont pas été reposées : même tuile

Ensuite : **0.110. La fonction `volumeSon()` — le volume général** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++